Politecnico di Milano

Ingegneria del Software – a.a. 2005/06

Appello del 16 febbraio 2007

Cognome

Nome

Matricola

Sezione (segnarne una) Baresi □, Ghezzi □, Morzenti □, SanPietro □

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare **solo i fogli distribuiti** utilizzando il **retro** delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. **non** verranno **in nessun caso** presi in considerazione. È possibile scrivere **in matita**.
3. È possibile consultare liberamente libri, manuali o appunti. È **proibito** l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 2h.
6. Punteggio totale a disposizione: 25/30. La sufficienza si raggiunge con 16 punti, a cui sommare il risultato delle prove di laboratorio.

Esercizio 1: .....

Esercizio 2: .....

Esercizio 3: .....

Esercizio 4: .....

Totale .....

Esercizio 1 (punti 10)

Il tipo di dato astratto (ADT) **Tombola** fornisce un'astrazione per una partita a tombola. Un giocatore estrae un numero usando il metodo **estrai()**. Questo metodo restituisce ogni volta un numero (diverso e a caso) tra 1 e 90. Se l'estrazione di un numero provoca una tombola, il metodo **estrai** deve generare un'eccezione **ExceptionTombola** e chiudere la partita. A ogni partita a tombola è associato un certo insieme di cartelle dei vari giocatori partecipanti; ogni cartella è rappresentata da un esemplare del tipo di dato astratto **Cartella**. L'ADT **Tombola** fornisce i metodi puri **tombola**, **cartelle** e **numeriEstratti**:

```
//@ requires true;
//@ ensures (*restituisce true se i numeri di c sono stati estratti tutti*) ;
public boolean /*@pure@*/ tombola(Cartella c)

//@ requires true;
//@ ensures (*restituisce le cartelle attive*) ;
public arrayList<Cartella> /*@pure@*/ cartelle()

//@ requires true;
//@ ensures (*restituisce la sequenza dei numeri estratti fino a quell' istante, in ordine cronologico di estrazione (quelli estratti per primi occupano le prime posizioni dell'arrayList)*) ;
public arrayList<Integer> /*@pure@*/ numeriEstratti()
```

a- Si scriva la postcondizione del metodo **estrai**, che restituisce il nuovo estratto:

```
//@ requires true;
public int estrai() throws ExceptionTombola
```

evidenziando che:

1. Il numero estratto è un numero non estratto in precedenza, compreso tra 1 e 90.
2. La sequenza dei numeri già estratti resta immutata, con l'unica aggiunta del numero estratto
3. Il numero estratto non causa una tombola.

SOLUZIONE

```
//@ ensures 1<=\result && \result <=90 && \old(!numeriEstratti().find(\result)) &&
numeriEstratti().lastElement() == \result &&
numeriEstratti.size() == \old(numeriEstratti.size()+1) &&
\forall(int i; 0<=i && i<numeriEstratti().size()-1);numeriEstratti.get(i) ==\old(numeriEstratti.get(i))
&&
\forall(int i; 0<=i && i< cartelle().size()); !cartelle.get(i).tombola())
```

b- Si scriva la clausola signals relativa al metodo **estrai**. In particolare, il metodo prevede che se il numero estratto provoca la tombola, questo viene comunque inserito nella sequenza dei numeri estratti.

```
//@ signals (ExceptionTombola e)
//alcune condizioni sono identiche a quelle della ensures:
numeriEstratti.size() == \old(numeriEstratti.size()+1) &&
\forall(int i; 0<=i && i< numeriEstratti().size()-1);numeriEstratti.get(i) ==\old(numeriEstratti.get(i))
&&
//ma serve anche indicare che l'ultimo elemento della sequenza dei numeri estratti è nuovo:

!exists( int i; i == numeriEstratti().lastElement(); \old(numeriEstratti().find(i)) &&
// e che una cartella fa tombola
\exists(int i; 0<=i && i<= cartelle().size()-1); cartelle.get(i).tombola()).
```

c- Sapendo che l'ADT **Cartella** fornisce il metodo puro per conoscere i numeri di una cartella:

```
//@ requires true ;
//@ ensures (*restituisce i numeri in this*) ;
public arrayList<Integer> /*@pure@*/ numeri()
```

Si scriva l'invariante pubblico per la classe **Tombola** per imporre che una tombola non può avere due cartelle contenenti gli stessi numeri e che le cartelle di ogni esemplare di **Tombola** devono coprire tutti i numeri da 1 a 90, ovvero ogni numero deve essere presente in almeno una cartella.

```
//@public invariant
//@(\forall int i; 0<=i && i< cartelle().size()-1;
        (\forall int j; i<j && j<= cartelle().size()-1;
                \exists(int k; 1<=k && k<=90;
                cartelle.get(i).numeri().find(k) && !cartelle.get(j). numeri().find(k)))) &&
//@(\forall int n; 1<= n && n<=90;
                (\exists int i; 0<=i && i< cartelle().size(); cartelle().get(i).numeri().find(n))));
```

Esercizio 2 (punti 6)

Si scriva in Java un iteratore che permetta di ottenere, in una sequenza casuale, tutti i numeri compresi tra 1 e 90 (si può immaginare che tale iteratore possa essere impiegato nell'implementazione del metodo **estrai** del precedente esercizio).
Nell'implementare l'iteratore si suggerisce di far uso dei seguenti dati

```
private boolean[] tabellone;
private int numEstratti ;
```

SI definisca sia il metodo **nuovaSequenza** che restituisce l'iteratore, sia la classe che definisce l'iteratore (ossia il generatore di numeri interi). L'interatore deve: (a) fornire ogni numero una sola volta, (b) marcare il numero estratto nell'array di booleani e (c) incrementare il contatore **numEstratti**. Si ipotizzi di poter disporre di un metodo statico **numero** che restituisce un numero casuale tra 1 e 90.

```
import java.util.Iterator;

public class Tabellone {

        public Iterator <Integer> nuovaSequenza() {
                return new SequenzaCasuale() ;
        }

        public static class SequenzaCasuale implements Iterator<Integer> {
                private boolean [] tabellone ;
                private int numEstratti ;

        SequenzaCasuale() {
           tabellone = new boolean[90] ;
           for (int i=0; i<90; i++) tabellone[i]=false;
           numEstratti = 0;
        }

        public boolean hasNext() {
                  return numEstratti<90;
        }

        public Integer next() throws NoSuchElementException {
                if (numeEstratti>=90) throw new NoSuchElementException("nuovaSequenza");

                numEstratti++ ;

                int n = numero();

                while (tabellone[n]) n = (n+1)%90;
                tabellone[n]=true ;
                return n;
        }
}
```

Esercizio 3 (punti 5)

Si fornisca l'invariante privato per la classe **Cartella** che fornisce un'astrazione della cartella della tombola. Si ricorda che (a) ogni cartella contiene 15 numeri, tutti diversi, fra 1 a 90 (b) i numeri sono disposti su 3 righe, (c) ogni riga contiene 5 numeri, (d) ogni cartella deve contenere almeno un numero per ognuna delle nove decine di cui è costituito l'insieme dei numeri estraibili.

```
public class Cartella {
        private int[][] num;
        ...
}
```

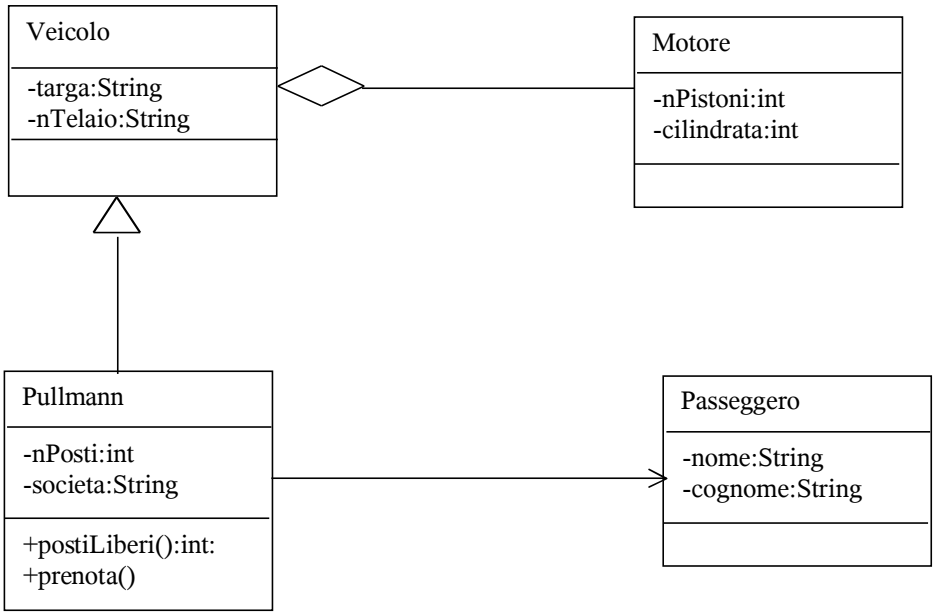
SOLUZIONE :

```
/* private invariant

num.length==3 &&
(\forall int k; 0<=k && k<3 ; num[k].length==5 ) &&
(\forall int i ; 0<=i  && i<=2 ;
        (\forall(int j ; 0<=j && j<=4 ; 1<=num[i][j] && num[i][j]<=90 &&
                !(\exists int h ; 0<=h && h<=2;
                        (\exists int k;  0<=k && k<=4 && i !=h && j !=k; num[i][j]==num[h][k])) &&
(\forall int n ; 0<=n  && n<=8 ;
        (\exists int h,k ; 0<=h && h<=2 && 0<=k && k<=4 ; num[h][k]/10 == n)) ;
```

Esercizio 4 (punti 4)

Si scrivano le parti dichiarative delle classi descritte dal seguente diagramma delle classi UML.
Si scriva anche la postcondizione del metodo “prenota” della classe Pullman.




```
package esame;

public class Veicolo {
	private String targa;
	private String nTelaio;
	private Motore motore;
}

public class Motore {
	private int nPistoni;
	private int cilindrata;
	private Veicolo veicolo;
}

public class Pullman extends Veicolo{
	private int nPosti;
	private String societa;
	private Passeggero[] passeggeri;

	public int postLiberi(){
		...
	}

	public void prenota(){
		…
	}

}

public class Passeggero {
```

```
        private String nome;
        private String cognome;
}
```

Parte B

```
//@ ensures postiLiberi() = \old(postLiberi()) -1;

public void prenota(){
        ...
}
```